

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Arquitectura de Computadoras

Nombre: Alonso Jaquez Sara Iriel

Maestro: Jorge Ernesto López Arce Delgado

Actividad 04 - Sumador Nibble y Sumador 8 bits

Sección: D03

I. Introducción

Dentro de la arquitectura de computadoras, uno de los componentes más importantes es la Unidad Aritmético Lógica (ALU), ya que es el bloque encargado de ejecutar las operaciones matemáticas y lógicas que permiten el funcionamiento de un procesador. Todas las operaciones complejas que realiza una computadora, desde cálculos simples hasta procesamiento gráfico avanzado, dependen en gran medida de operaciones aritméticas básicas, siendo la suma la más fundamental de todas.

En la presente práctica se desarrollaron distintos tipos de sumadores utilizando el lenguaje de descripción de hardware Verilog. Se implementaron dos versiones de los sumadores: una estructural, basada en la instancia de módulos previamente diseñados, y otra comportamental, utilizando operadores aritméticos propios del lenguaje. Se trabajó con sumadores de 4 bits y 8 bits.

Además del desarrollo práctico, se realizó una investigación teórica sobre los conceptos de complemento a 1, complemento a 2 y representación en punto flotante, los cuales son esenciales para comprender cómo los sistemas representan números digitales negativos y reales. Esta combinación entre teoría y práctica permite entender no solo cómo funciona un sumador, sino también su importancia dentro de la ALU y del procesador en general.

II. Objetivos

El objetivo principal de esta actividad es comprender que la suma es la operación aritmética más importante dentro de los sistemas digitales, ya que a partir de ella se pueden implementar otras operaciones como la resta mediante complemento a 2. De manera específica, se busca:

- a) Implementar un sumador completo utilizando módulos previamente diseñados.
- b) Implementar una versión comportamental de los sumadores.
- c) Verifique el funcionamiento mediante simulación en ModelSim.
- d) Visualizar los resultados en formato decimal (base decimal).

III. Marco teórico

La suma como operación fundamental

En los sistemas digitales, todas las operaciones aritméticas pueden descomponerse en sumas. Incluso la resta se realiza internamente como una suma utilizando el complemento a 2. Por esta razón, el diseño de sumadores eficientes es un aspecto clave en la arquitectura de procesadores.

La ALU contiene circuitos especializados para realizar operaciones aritméticas y lógicas. Entre estos circuitos, el sumador es el bloque base sobre el cual se construyen otros módulos más complejos.

-Complemento a 1: Es un método para representar números negativos en binario. Se obtiene invirtiendo todos los bits del número original.

Ejemplo:

Número decimal: 6

Binario (4 bits): 0110

Complemento a 1: 1001

Este método permite representar números negativos, pero presenta una desventaja importante: existen dos representaciones para el cero (0000 y 1111). Esta ambigüedad complica las operaciones aritméticas dentro del hardware.

Aunque actualmente no es el método más utilizado, es importante estudiarlo porque sirve como base para comprender el complemento a 2.

-Complemento a 2: Es el método estándar utilizado en la mayoría de los procesadores modernos. Se obtiene aplicando complemento a 1 y posteriormente sumando 1 al resultado.

Proceso:

1. Se invierten todos los bits.
2. Se suma 1 al resultado.

Ejemplo:

Número decimal: 6

Binario: 0110

Complemento a 1: 1001 + 1

Resultado: 1010

Las ventajas principales son:

1. Solo existe una representación del cero.
2. Permite realizar restas utilizando sumadores.
3. Simplifica el diseño del hardware.

Gracias al complemento a 2, la ALU no necesita un circuito separado para reiniciar; simplemente suma el complemento del número.

-Representación en Punto Flotante: No todos los números que procesan una computadora son enteros. Para representar números reales se utiliza el formato de punto flotante, definido principalmente por el estándar IEEE 754.

Este formato divide el número en tres partes:

1. Signo
2. Exponente
3. Mantisa

La representación en punto flotante permite manejar valores muy grandes o muy pequeños con un nivel de precisión adecuado. Es ampliamente utilizado en aplicaciones científicas, simulaciones, gráficos y procesamiento en GPU.

-Tipos de Sumadores

Existen distintas arquitecturas de sumadores, entre las más importantes se encuentran:

- Sumador de acarreo de ondulación (RCA)
- Sumador de acarreo anticipado (CLA)
- Sumador de selección de acarreo
- Llevar a salvo Adder

El Ripple Carry Adder es el más sencillo y consiste en conectar varios sumadores completos en cascada. Su principal desventaja es el tiempo de propagación del acarreo.

Las CPU modernas utilizan variantes más rápidas como Carry Lookahead para reducir el tiempo de cálculo.

IV. Desarrollo

Para el desarrollo de esta práctica se utilizó el lenguaje Verilog y el simulador ModelSim. Se implementaron los módulos solicitados siguiendo un diseño jerárquico y modular.

1. Medio Sumador (HA)

El medio sumador es el circuito más básico para realizar una suma binaria. Recibe dos entradas (A y B) y produce dos salidas:

1. Suma (XOR)
2. Acarreo (Y)

Este módulo se implementó utilizando asignaciones continuas, lo que permite describir directamente la lógica combinacional. El medio sumador es importante porque sirve como bloque base para el sumador completo.

2. Sumador Completo (FA)

El sumador completo amplía el funcionamiento del medio sumador al incluir un acarreo de entrada (Ci).

En esta práctica, el módulo fue implementado utilizando dos medios sumadores y una compuerta O para combinar los acarreos.

Funcionamiento general:

1. Se suman A y B.
2. Se suma el resultado con Ci.
3. Se combinan los acarreos generados.

Este diseño demuestra claramente la instancia de módulos previamente hechos, cumpliendo con los requisitos de la actividad.

3. Sumador de 4 Bits (Instanciado)

El módulo de 4 bits fue implementado utilizando cuatro sumadores completos conectados. Cada sumador recibe el acarreo generado por el anterior.

4. Sumador de 8 Bits (Instanciado)

El sumador de 8 bits fue implementado utilizando dos módulos de 4 bits conectados. El acarreo de salida del primer bloque de 4 bits se conecta como entrada del segundo bloque.

5. Implementación Comportamental

Además del diseño estructural, se implementaron sumadores utilizando la descripción comportamental.

Se utilizó la siguiente asignación:

asignar {Cout, C} = A + B + Cin;

Este tipo de implementación permite describir la operación de manera más directa, dejando que el sintetizador determine la estructura interna.

La diferencia principal entre ambos enfoques es:

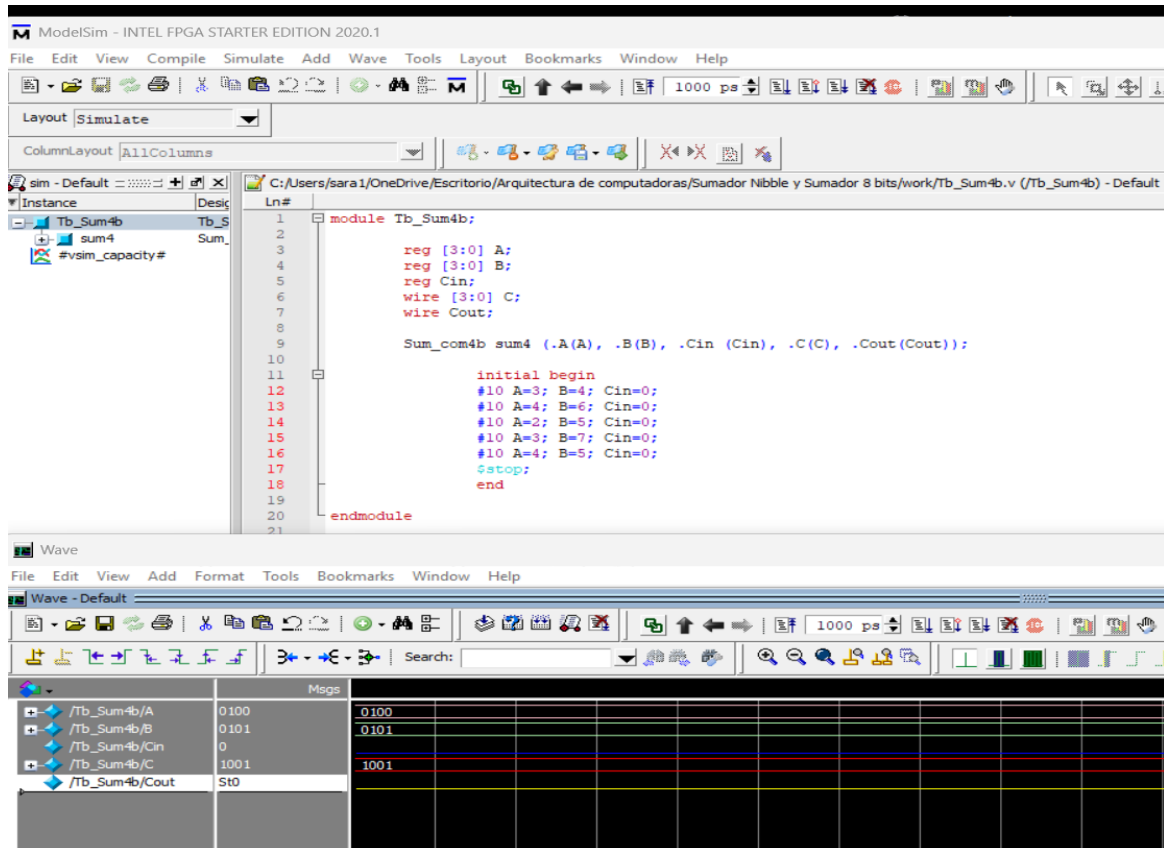
Estructural: muestra cómo está construido el circuito.

Comportamental: describe qué hace el circuito.

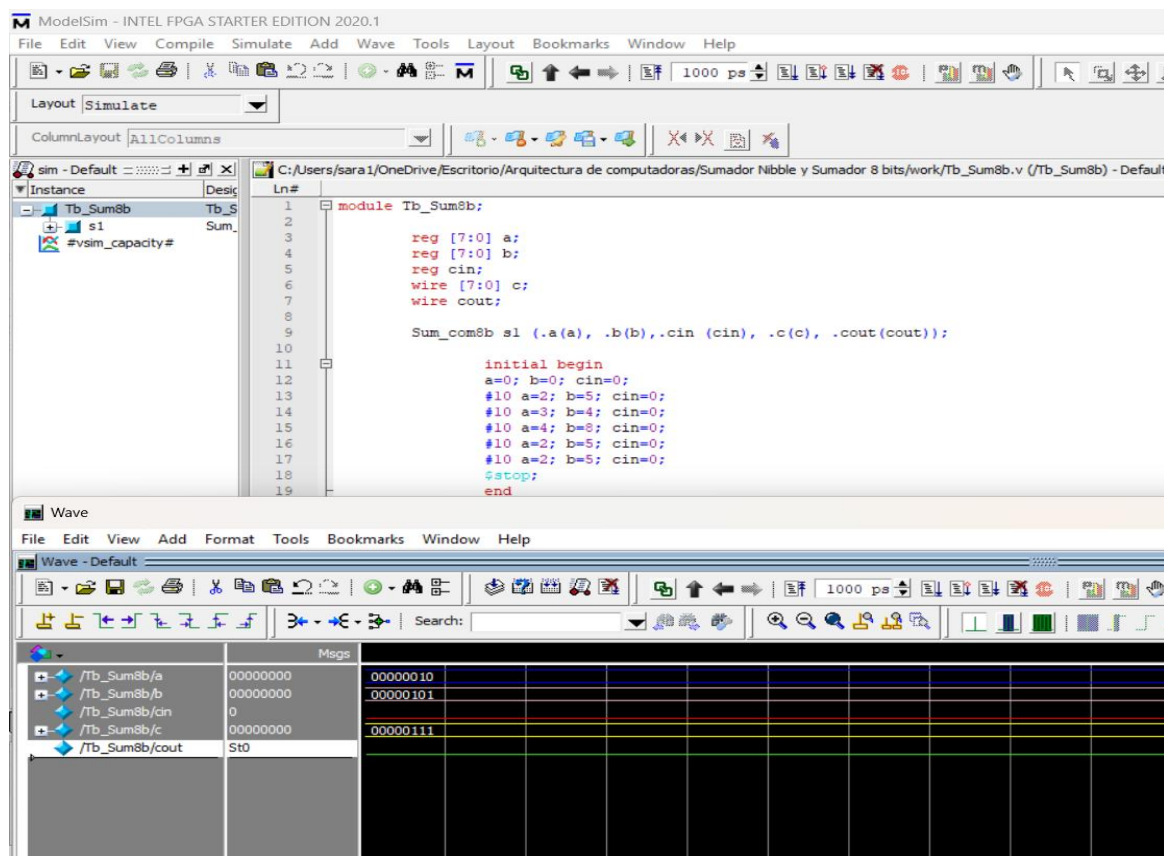
6. Simulación

Se desarrollaron dos bancos de pruebas:

Tb_Sum4b



Tb_Sum8b



En cada uno se realizaron cinco sumas distintas, cumpliendo con los requisitos de la actividad.

Durante la simulación en ModelSim se configuró la visualización en formato decimal, lo que permitió verificar claramente que las sumas eran correctas.

La simulación permitió validar el funcionamiento tanto del diseño estructural como del comportamental.

V. Conclusión

La práctica permitió comprender que la suma es la operación base de los sistemas digitales y que a partir de ella se pueden construir operaciones más complejas. El diseño del medio sumador y del sumador completo permitió entender cómo se genera la suma bit a bit y cómo se propaga el acarreo.

La implementación estructural reforzó el concepto de diseño modular y jerárquico en Verilog, mientras que la implementación comportamental mostró una forma más compacta y eficiente de describir la misma operación.

Finalmente, se comprobó mediante simulación que ambos enfoques funcionan correctamente, lo cual demuestra la importancia de la verificación en el diseño digital.

VI. Referencias

TylerMSFT. (s. f.). *Representación de punto flotante de IEEE*. Microsoft Learn. <https://learn.microsoft.com/es-es/cpp/build/ieee-floating-point-representation?view=msvc-170> Signo y magnitud, complemento a 1 y a 2 y el exceso Z para tontos. (2020, 17 diciembre). *Odisea Geek*. <https://odiseageek.es/posts/signo-y-magnitud-complemento-a-1-y-a-2-y-el-exceso-z-para-tontos/>

Kohli, V. (2025, 3 junio). *Half Adder and Full Adder - Truth Table, Circuit, and Working*. Electronics For You – Official Site ElectronicsForU.com. <https://www-electronicsforu-com.translate.goog/technology-trends/learn-electronics/half-adder-full-adder-truth-table-circuit-working-kmap? x tr sl=en& x tr tl=es& x tr hl=es& x tr pto=tc>

Así es cómo tu CPU y GPU realizan los cálculos matemáticos en tu PC -. (2021, 16 junio). <https://goldenti.com/site/asi-es-como-tu-cpu-y-gpu-realizan-los-calculos-matematicos-en-tu-pc/>

El chisme de Spectre y Meltdown: Cuando los procesadores quedaron exhibidos.

Durante muchos años, los procesadores modernos fueron considerados piezas casi perfectas de la ingeniería informática. Eran rápidos, eficientes y, sobre todo, confiables. Sin embargo, en 2018 salió a la luz uno de los escándalos más grandes en la historia de la computación moderna: las vulnerabilidades conocidas como Spectre y Meltdown. Este descubrimiento sacudió la industria tecnológica y dejó claro que, detrás de tanta velocidad, había secretos incómodos.

El problema surgió a partir de una técnica ampliamente utilizada en los procesadores modernos llamada ejecución especulativa. Esta técnica permite que el procesador se adelante a ejecutar instrucciones antes de saber si realmente serán necesarias, con el objetivo de ahorrar tiempo y mejorar el rendimiento. El chisme comienza cuando se descubrió que ese “adelantarse” podía ser manipulado para acceder a información que debería estar completamente protegida.

En otras palabras, el procesador hacía cosas “de más”, y alguien con malas intenciones podía aprovecharse de eso para espiar datos sensibles.

Meltdown: El que cruzó la línea.

Meltdown fue la vulnerabilidad más directa y escandalosa. Básicamente, rompía la separación entre las aplicaciones y el sistema operativo, una de las reglas más importantes de la seguridad informática. Gracias a Meltdown, un programa común podía leer información almacenada en la memoria del sistema, como contraseñas o datos privados.

Este fallo afectó principalmente a procesadores Intel, lo que generó una fuerte polémica, ya que estos chips se usan en millones de computadoras alrededor del mundo. Fue como descubrir que cualquiera podía entrar sin permiso a una zona que se suponía altamente restringida.

Spectre: el manipulador silencioso.

Si Meltdown fue descubierto, Spectre fue mucho más astuto. Esta vulnerabilidad no rompía las reglas directamente, sino que engañaba al procesador para que, por su cuenta, revelara información confidencial.

Spectre aprovecha la manera en que los procesadores predicen el comportamiento de los programas. Al manipular esas predicciones, un atacante puede hacer que el procesador acceda a datos privados y deje rastros que luego pueden ser analizados. Lo más preocupante es que Spectre afecta a procesadores de Intel, AMD y ARM, es decir, prácticamente a todos.

Además, Spectre es difícil de eliminar por completo, ya que está relacionado con el diseño mismo de los procesadores modernos.

El escándalo en la industria.

Cuando este chisme salió a la luz, las reacciones no se hicieron esperar. Grandes empresas tecnológicas entraron en modo emergencia y comenzaron a lanzar parches de seguridad.

Entre las consecuencias más importantes se encuentran:

1. Actualizaciones urgentes de sistemas operativos como Windows, macOS y Linux
2. Cambios en navegadores web para reducir riesgos
3. Disminución del rendimiento en algunos equipos tras aplicar los parches
4. Rediseño de procesadores en generaciones posteriores

Este caso dejó una lección clara: la búsqueda de mayor velocidad había puesto en riesgo la seguridad de los usuarios.

Kocher, P., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., & Yarom, Y. (2018, January 3). *Spectre Attacks: exploiting speculative execution*. arXiv.org. <https://arxiv.org/abs/1801.01203>

Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., & Hamburg, M. (2018). *Meltdown: Reading Kernel Memory from User Space*. USENIX. <https://www.usenix.org/conference/usenixsecurity18/presentation/lipp>

<https://medium.com/@mattklein123/meltdown-spectre-explained-6bc8634cc0c2>